UCAM

Tema 3

Programación con Jetpack Compose

Aplicaciones para Dispositivos Móviles

Daniel Valero Carreras

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1. Introducción Jetpack Compose
2. Arquitectura básica
3. Componentes

1. Introducción Jetpack Compose

Definición

Imagen 1
Título asignado a la imagen



Jetpack Compose

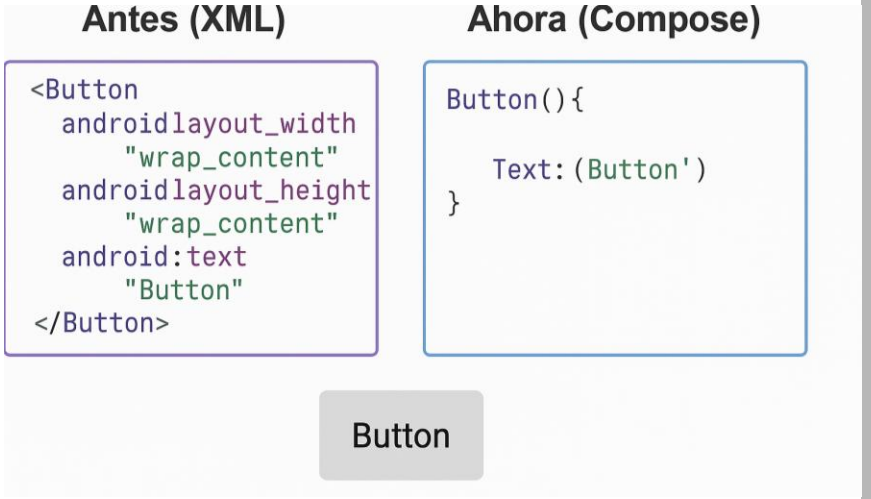
Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

1. Introducción Jetpack Compose

¿Qué es Jetpack Compose y por qué sustituye XML?

- Anteriormente, las vistas se diseñaban en XML
- Cada pantalla era un activity
- Desarrollado por Google como **sustituto de XML**.
- **Ventajas**:
  - Menos código
  - Mas flexible
  - Mejor integración con Kotlin
  - Reutilización y testing

Imagen 2
Título asignado a la imagen

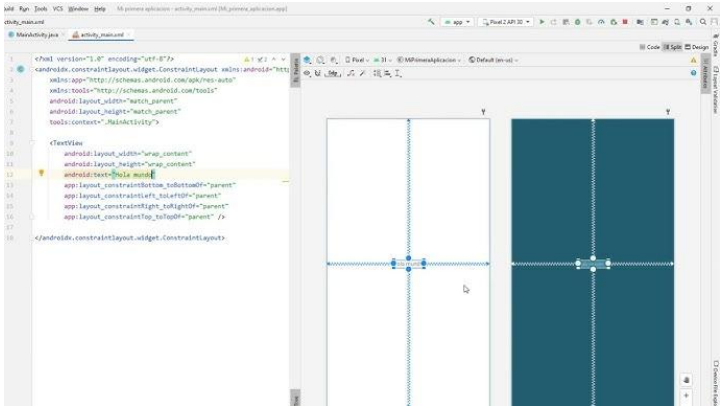


Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

1. Introducción Jetpack Compose

¿Qué es Jetpack Compose y por qué sustituye XML?

- Toolkit moderno y declarativo para **crear interfaces nativas en Android**.
- Desarrollado por Google como **sustituto de XML**.



```
class MainActivity : AppCompatActivity() {

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    bu1.setOnClickListener({
        val intent = Intent(this, Main2Activity::class.java)
        startActivities(intent)
    })

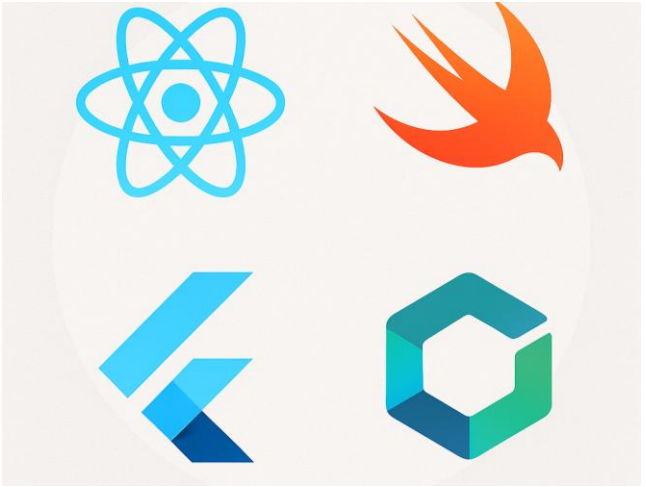
}
}
```

1. Introducción Jetpack Compose

Relación con otros frameworks declarativos

- Mismo paradigma que otros frameworks modernos:
  - React (JavaScript → Web)
  - Flutter (Dart → Multiplataforma)
  - SwiftUI (Swift → iOS/macOS)
- Enfoque **UI = función de estado** → "Describe qué quieres ver, no cómo dibujarlo paso a paso"

Imagen 2
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

2. Arquitectura básica

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

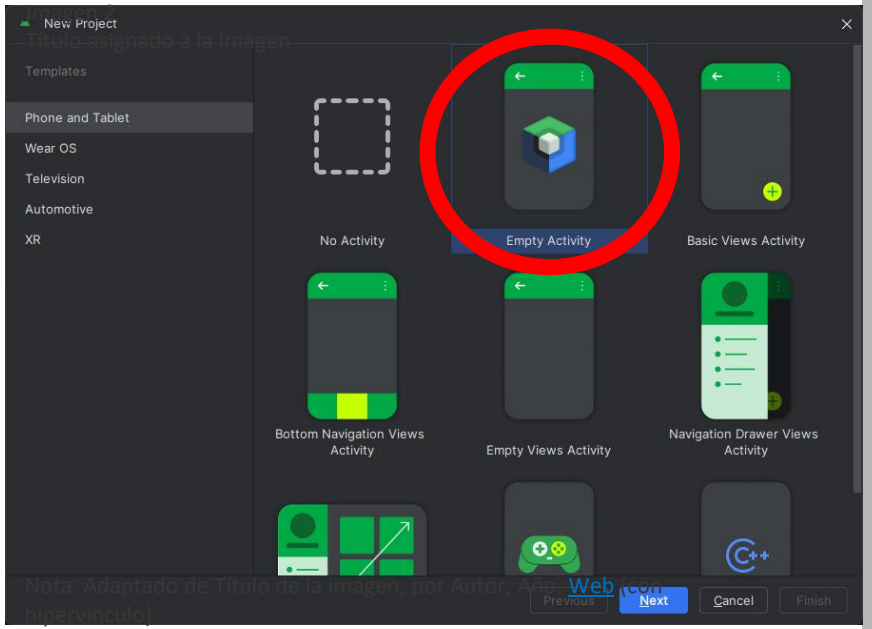
2. Arquitectura básica

Crear un proyecto Compose

- Abrir Android Studio
  - Clic en **“New Project”**.
- Elegir plantilla
  - Seleccionar **“Empty Activity”** (no elegir las de XML).
  - Esta plantilla ya viene configurada con Compose.

Imagen 2

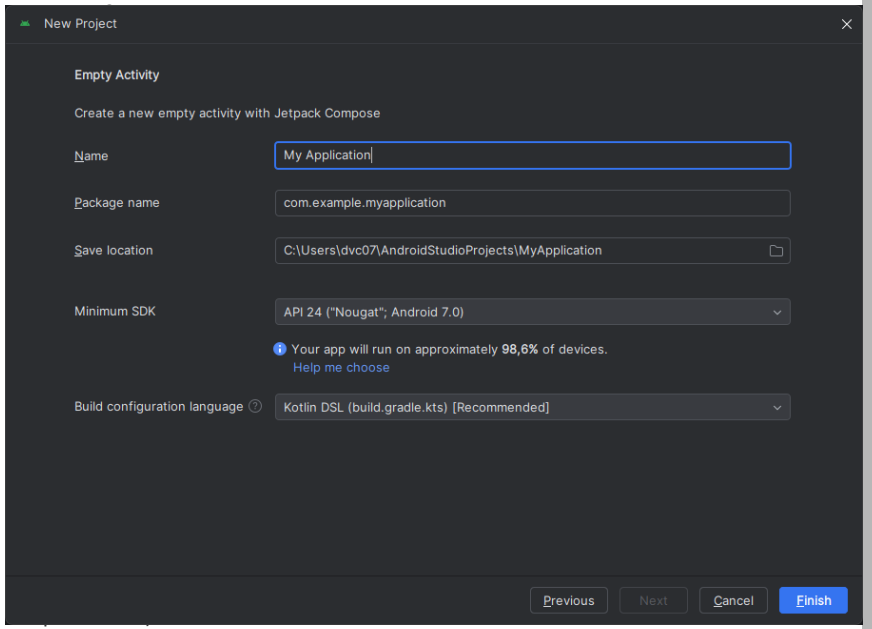
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año. Web (con hipervínculo)

2. Arquitectura básica
Crear un proyecto Compose

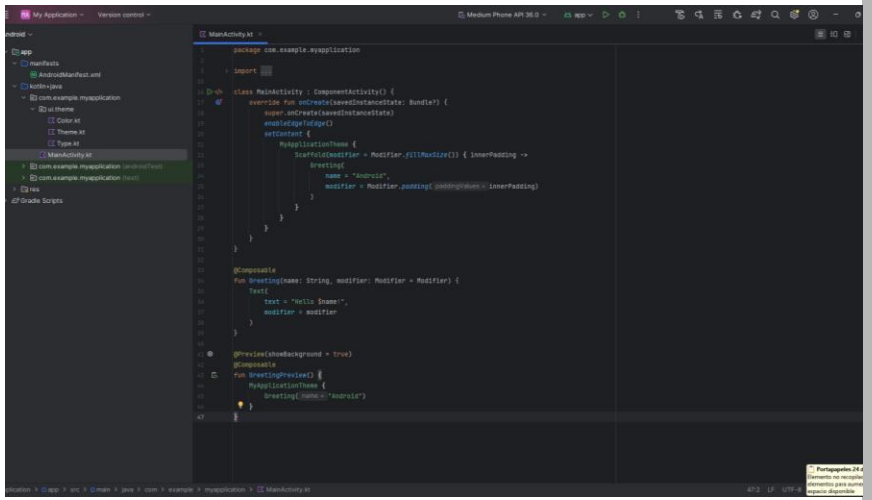
- Configurar el proyecto
  - **Nombre de la aplicación** (ej: MiPrimeraAppCompose).
  - **Nombre del paquete** (ej: com.ejemplo.miapp).
  - **Lenguaje**: Kotlin.
  - **Versión mínima de SDK**: normalmente API 24 (Android 7.0) o superior.



2. Arquitectura básica
Crear un proyecto Compose

- Estructura inicial del proyecto
  - **MainActivity.kt** → punto de entrada de la app.
  - Función **setContent {}** donde se define la UI con Compose.
  - Paquete ui.theme con temas, colores y tipografías configurados.

Imagen 2
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

2. Arquitectura básica
Preview

- Podemos previsualizar los composables
    - **Etiqueta preview**
- Ahorra tiempo → no tienes que compilar y lanzar la app cada vez
- Iteración rápida en la UI → puedes probar modificaciones visuales al instante.

Imagen 2
Título asignado a la imagen

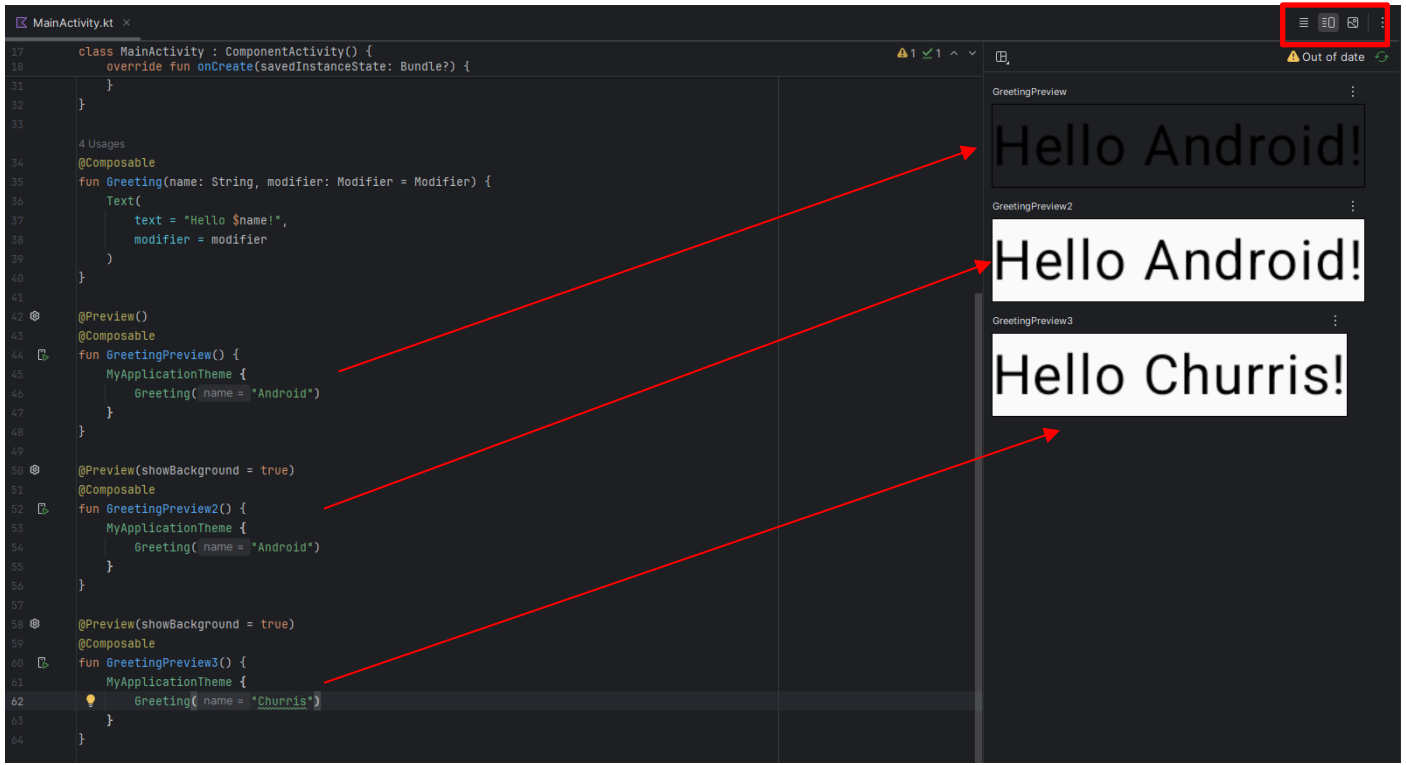
```
@Composable
fun Greeting(name: String) {
    Text("Hola $name!")
}

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun GreetingPreview() {
    Greeting("Mundo")
}
```

Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

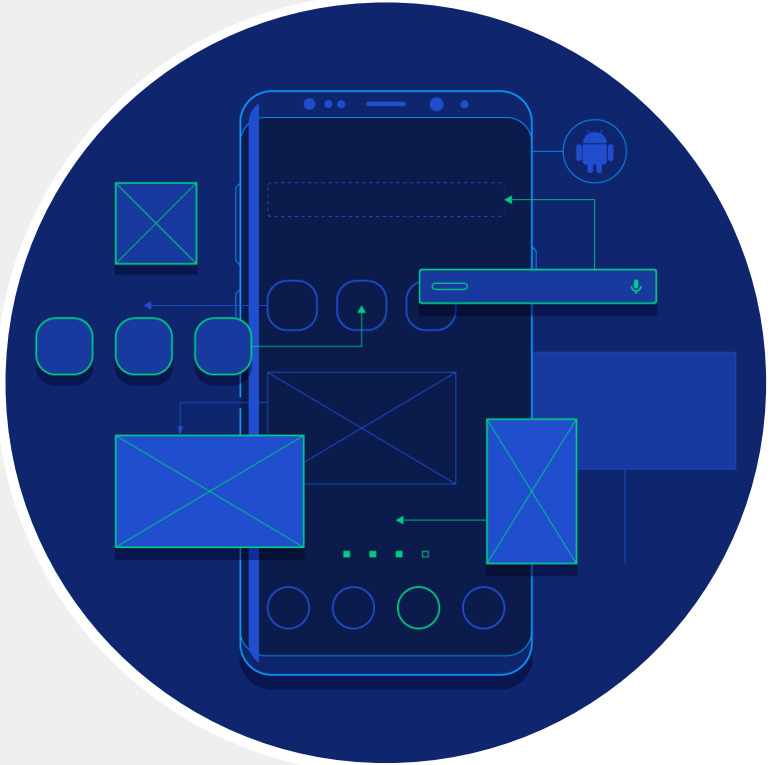
2. Arquitectura básica

Preview



3. Componentes

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

3. Componentes
Composable

- Una función que describe una parte de la interfaz de usuario
- Sustituye a los Views
- Características:
  - **Declarativo** → describes la UI en lugar de imperativamente modificarla.
  - **Reutilizable** → puedes componer (de ahí el nombre) funciones dentro de otras.
  - **Eficiente** → Compose decide qué partes deben volver a dibujarse cuando cambia el estado.
  - **Jerárquico** → la UI se construye combinando Composables más pequeños dentro de otros más grandes

Imagen 2
Título asignado a la imagen

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hola $name!")
}
```

Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

3. Componentes
Modifier

- Es la forma en la que le dices a un Composable **cómo debe verse, cómo debe comportarse y cómo debe colocarse** dentro de la UIEs la configuración de la visión de ese composable
- Ajusta la apariencia
  - Padding, aling, background, etc.
- Añade interacciones
  - Ej: clickable

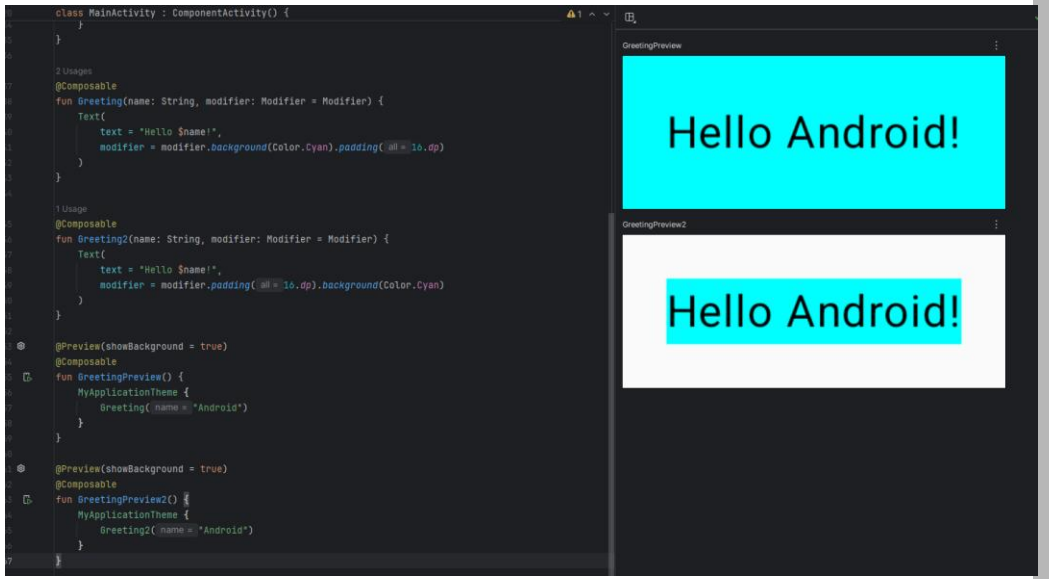
Imagen 2
Título asignado a la imagen

```
@Composable
fun GreetingCard() {
    Text(
        text = "Hola Jetpack Compose!",
        modifier = Modifier
            .padding(16.dp)        // margen interno
            .background(Color.Yellow) // fondo amarillo
            .clickable { println("Texto clicado") } // acción al tocarlo
    )
}
```

Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

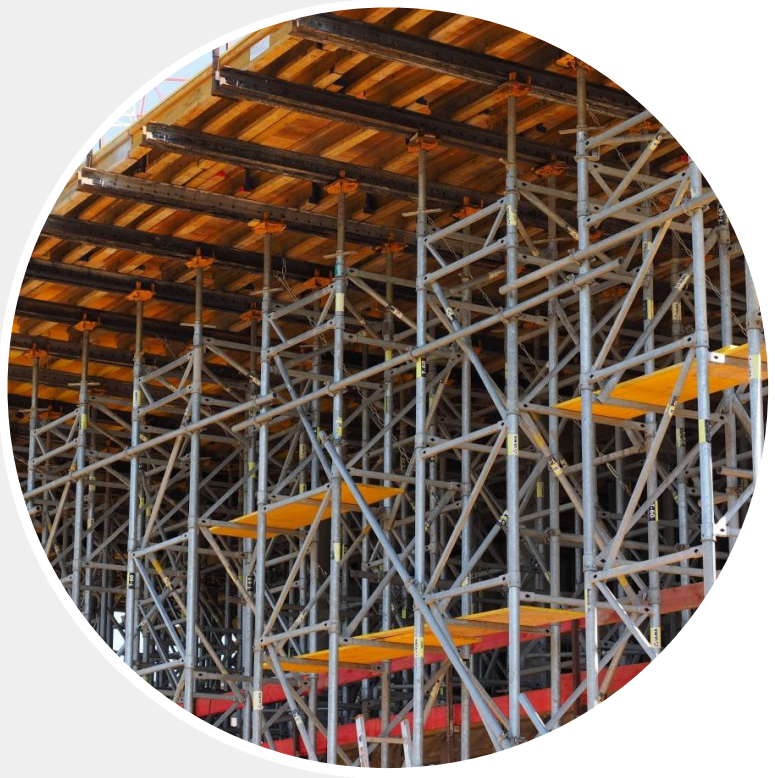
3. Componentes
Modifier

- El orden de los atributos importa
- Observa la diferencia:
  - padding.background
  - background.padding



3.1. Layouts

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

3. Componentes
Layouts

- Los layouts son contenedores que sirven para estructurar los composables
  - **Box**: superpone elementos (uno encima de otro) o los coloca usando alineación dentro de un solo contenedor.
  - **Column**: coloca los elementos **uno debajo del otro** (en vertical).
  - **Row**: coloca los elementos **uno al lado del otro** (en horizontal).
  - **ConstraintLayout** (librería de soporte) → Similar al clásico ConstraintLayout de Android Views, con restricciones más complejas.
  - **Scaffold** → Proporciona la estructura típica de una pantalla con TopAppBar, BottomBar, FloatingActionButton, y contenido principal.

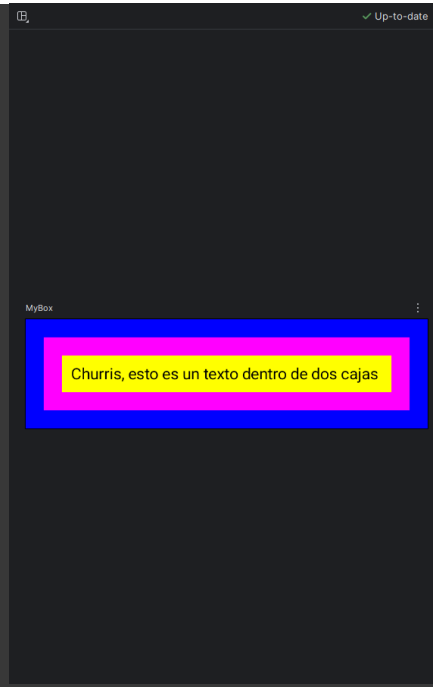
3. Componentes
Box

- Es un contenedor (layout) que permite **apilar y superponer elementos** en la interfaz de usuario.
- Coloca a sus hijos uno encima del otro (en orden de declaración).
- Permite controlar la alineación de cada hijo dentro del Box con el modificador Modifier.align().
- Ejemplos usos:
  - Un icono encima de una imagen.
  - Un texto centrado dentro de un área.
  - Un botón flotante sobre otro contenido.

3. Componentes

Box

```
@Preview(showBackground = true)
@Composable
fun MyBox() {
    Box(modifier = Modifier
        .background(Color.Blue)
        .padding(16.dp), contentAlignment = Alignment.Center) {
        Box(modifier = Modifier
            .background(Color.Magenta)
            .padding(16.dp), contentAlignment = Alignment.Center) {
            Text(text = “Churris, esto es un texto dentro de dos cajas",
                modifier = Modifier.background(Color.Yellow).padding(8.dp))
        }
    }
}
```

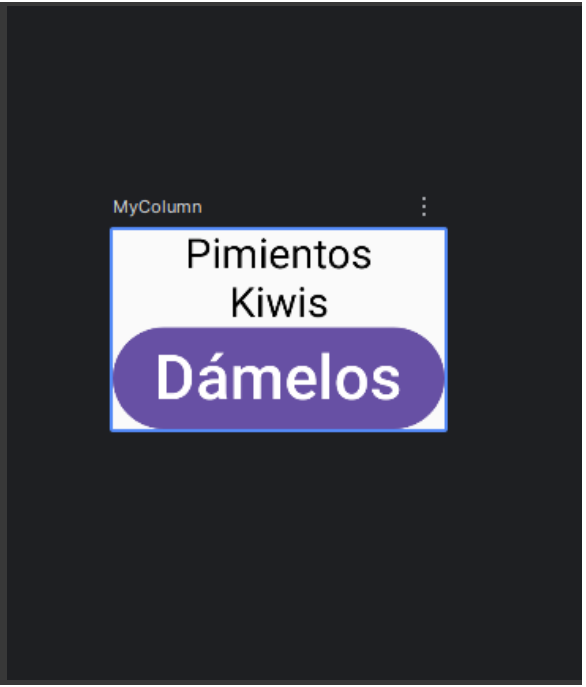


3. Componentes
Column

- Es un layout que organiza sus elementos **de arriba hacia abajo**, es decir, en una **disposición vertical**.
- Cada hijo se coloca **debajo del anterior**.
- Puedes controlar cómo se alinean horizontal y verticalmente sus elementos con parámetros como:
  - horizontalAlignment → alineación en el eje horizontal (izquierda, centro, derecha).
  - verticalArrangement → espaciado y orden en el eje vertical.

3. Componentes
Column

```
@Preview(showBackground = true)
@Composable
fun MyColumn() {
    Column(modifier = Modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Pimientos", fontSize = 24.sp)
        Text("Kiwis", fontSize = 24.sp)
        Button(onClick = { }) {
            Text("Dámelos", fontSize = 36.sp)
        }
    }
}
```



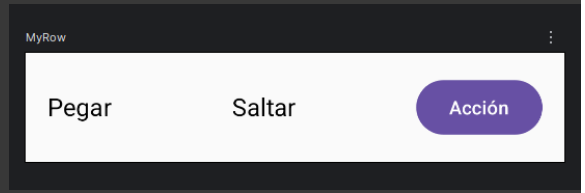
3. Componentes
Row

- Es un layout que organiza sus elementos **de izquierda a derecha**, es decir, en una **disposición horizontal**.
- Coloca cada hijo al **lado del anterior** en una fila.
- Puedes controlar cómo se alinean horizontal y verticalmente sus elementos con parámetros como:
  - horizontalAlignment → cómo se distribuyen los elementos en el eje horizontal (espaciados, centrados, alineados al inicio o al final)
  - verticalArrangement → cómo se alinean en el eje vertical (arriba, centro, abajo).

3. Componentes

Row

```
@Preview(showBackground = true)
@Composable
fun MyRow() {
    Row(
    modifier = Modifier
    .fillMaxWidth()
    .padding(16.dp),
    horizontalArrangement = Arrangement.SpaceBetween,
    verticalAlignment = Alignment.CenterVertically
    ) {
        Text("Pegar", fontSize = 18.sp)
        Text("Saltar", fontSize = 18.sp)
        Button(onClick = { }) {
            Text("Acción")
        }
    }
}
```



3. Componentes
ConstraintLayout

- Puedes **alinear** un elemento respecto a otro (ej: a la izquierda de un botón, centrado con respecto a otro texto).
- Permite crear **layouts complejos** sin necesidad de anidar muchos Row, Column o Box.
- Muy útil cuando quieres que tu UI sea **responsiva** y con posiciones más precisas.
- Requiere **agregar la dependencia** del ConstraintLayout para Compose en tu proyecto.

3. Componentes

¿Cómo importar la librería ConstraintLayout?

- En build.gradle(app)

```
[dependencies]
implementation(libs.constraint.layout)
```

- En libs.versions.toml

```
[versions]
constraintVersion = "1.1.1“
[libraries]
constraint-layout = {module = "androidx.constraintlayout:constraintlayout-compose", version.ref = "constraintVersion" }
```

- Sincroniza el proyecto

3. Componentes
ConstraintLayout

- Métodos clave dentro de un Constraint

  top.linkTo(...) → conecta la parte superior de un elemento con otro (o con el contenedor).

  bottom.linkTo(...) → conecta la parte inferior.

  start.linkTo(...) / end.linkTo(...) → conectan lados izquierdo/derecho

- Hay que crear una variable de referencia

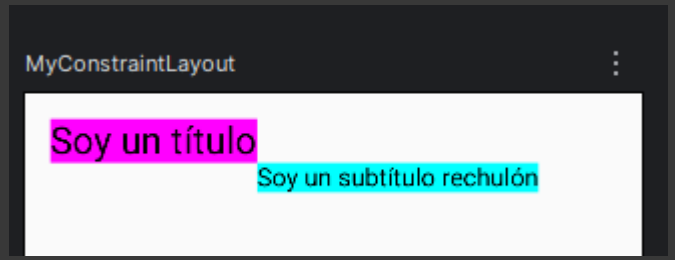
  createRefs()

3. Componentes
ConstraintLayout

```
@Preview(showBackground = true)
@Composable
fun MyConstraintLayout() {
   ConstraintLayout(
      modifier = Modifier.fillMaxSize()
   ) {
      val (titulo, subtitulo) = createRefs()  # identificador de las distintas regiones de nuestro layout que van a estar unidas
      Text(
         text = "Soy un título", fontSize = 24.sp, modifier = Modifier
            .constrainAs(titulo)  # asignación con la variable anterior
            top.linkTo(parent.top, margin = 16.dp)  #la región superior de este text está linkada con la parte superior de la app y un margen
            start.linkTo(parent.start, margin = 16.dp) )  #el inicio de este text (parte superior izquierda está linkada con la parte superior de la app y un margen}
            .background(Color.Magenta))
      Text(
         text = "Soy un subtítulo rechulón",
         fontSize = 16.sp, modifier = Modifier.constrainAs(subtitulo) {
            top.linkTo(titulo.bottom)
            start.linkTo(titulo.end)}
            .background(Color.Cyan)
      ) }
}
```

3. Componentes
ConstraintLayout

```
@Preview(showBackground = true)
@Composable
fun MyConstraintLayout() {
   ConstraintLayout(
      modifier = Modifier.fillMaxSize()
   ) {
      val (titulo, subtitulo) = createRefs()
      Text(
         text = "Soy un título", fontSize = 24.sp, modifier = Modifier
            .constrainAs(titulo) {
            top.linkTo(parent.top, margin = 16.dp)
            start.linkTo(parent.start, margin = 16.dp)}
            .background(Color.Magenta))
      Text(
         text = "Soy un subtítulo rechulón",
         fontSize = 16.sp, modifier = Modifier.constrainAs(subtitulo) {
            top.linkTo(titulo.bottom)
            start.linkTo(titulo.end)}
            .background(Color.Cyan)
      ) }
}
```



3. Componentes
Scaffold

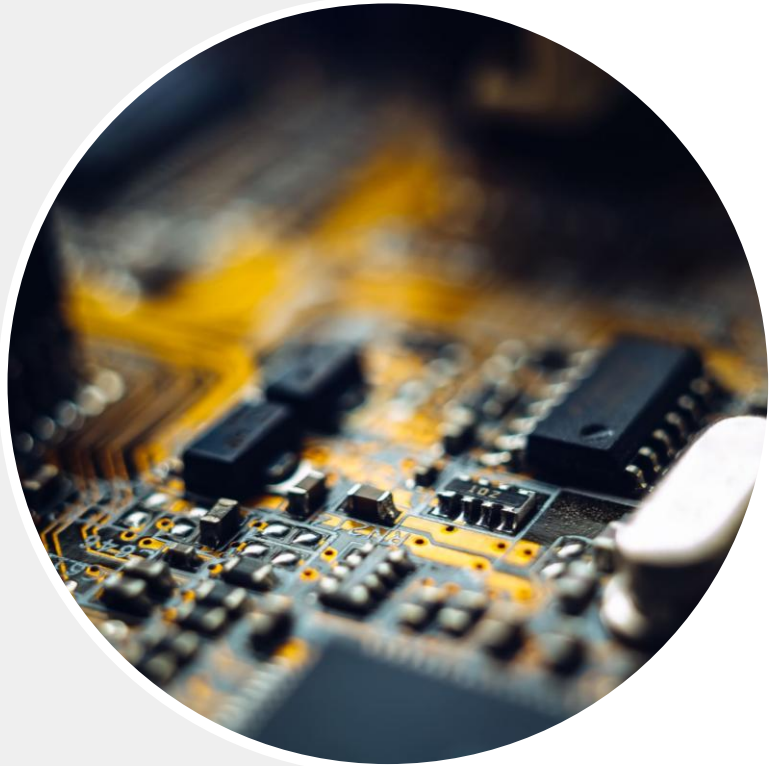
- Es el composable que define la estructura básica de la pantalla
- Es el “esqueleto” de la UI
- Incluye:
  - topBar → Barra superior (ej. TopAppBar).
  - bottomBar → Barra inferior (ej. navegación).
  - floatingActionButton → Botón flotante de acción principal.
  - drawerContent → Menú lateral (navigation drawer).
  - content → El contenido principal de la pantalla.

3. Componentes
Scaffold

- Es el composable que define la estructura básica de la pantalla
- Es el “esqueleto” de la UI
- Incluye:
  - topBar → Barra superior (ej. TopAppBar).
  - bottomBar → Barra inferior (ej. navegación).
  - floatingActionButton → Botón flotante de acción principal.
  - drawerContent → Menú lateral (navigation drawer).
  - content → El contenido principal de la pantalla.

3.2. Estados

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

3. Componentes
Estados

- Los estados son una forma de guardar y recordar valores que pueden cambiar con el tiempo y que afectan cómo se dibuja la interfaz.
  - Un estado es un valor observable.
  - Cuando ese valor cambia, Compose vuelve a componer (re-dibujar) solo las partes de la UI que lo usan.
  - Esto permite que la interfaz reaccione dinámicamente a cambios de datos.
- Tipos:
  - **remember { mutableStateOf(...) }** → guarda un valor que cambia mientras el composable existe en memoria.
  - **rememberSaveable { ... }** → similar, pero además sobrevive a cambios de configuración (como rotación de pantalla).

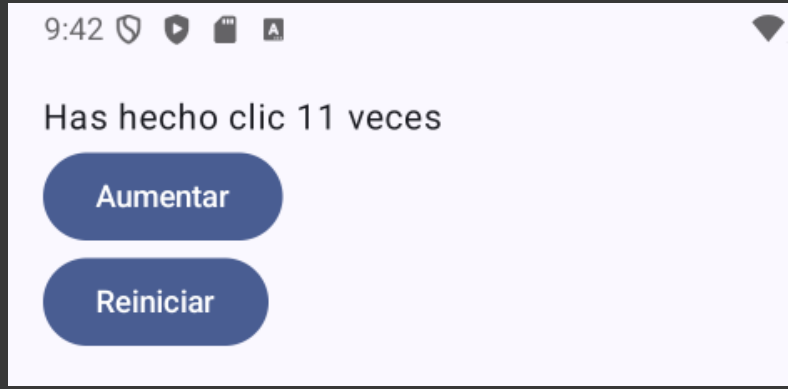
3. Componentes
Estados

- Los estados son una forma de guardar y recordar valores que pueden cambiar con el tiempo y que afectan cómo se dibuja la interfaz.
  - Un estado es un valor observable.
  - Cuando ese valor cambia, Compose vuelve a componer (re-dibujar) solo las partes de la UI que lo usan.
  - Esto permite que la interfaz reaccione dinámicamente a cambios de datos.
- Tipos:
  - **remember { mutableStateOf(...) }** → guarda un valor que cambia mientras el composable existe en memoria.
  - **rememberSaveable { ... }** → similar, pero además sobrevive a cambios de configuración (como rotación de pantalla).

3. Componentes

Estados (= remember)

```
@Composable
fun MyContadorState(modifier: Modifier) {
   // Estado que recuerda el valor del contador
   var contador = remember { mutableStateOf(0) }
   Column(
      modifier = modifier.padding(16.dp),
   ) {
      Text(text = "Has hecho clic ${contador.value} veces")
      Button(onClick = { contador.value +=1 }) {
         Text("Aumentar")
      }
      Button(onClick = { contador.value = 0 }) {
         Text("Reiniciar")
      }
   }
}
```

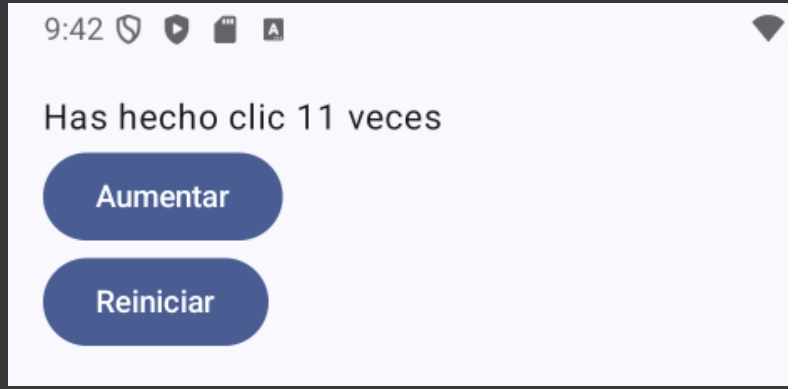


3. Componentes

Estados (by remember)

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue

@Composable
fun MyContadorState(modifier: Modifier) {
   // Estado que recuerda el valor del contador
   var contador by remember { mutableStateOf(0) }
   Column(
      modifier = modifier.padding(16.dp),
   ) {
      Text(text = "Has hecho clic $contador veces")
      Button(onClick = { contador +=1 }) {
         Text("Aumentar")
      }
      Button(onClick = { contador= 0 }) {
         Text("Reiniciar")
      }
   }
}
```



3. Componentes

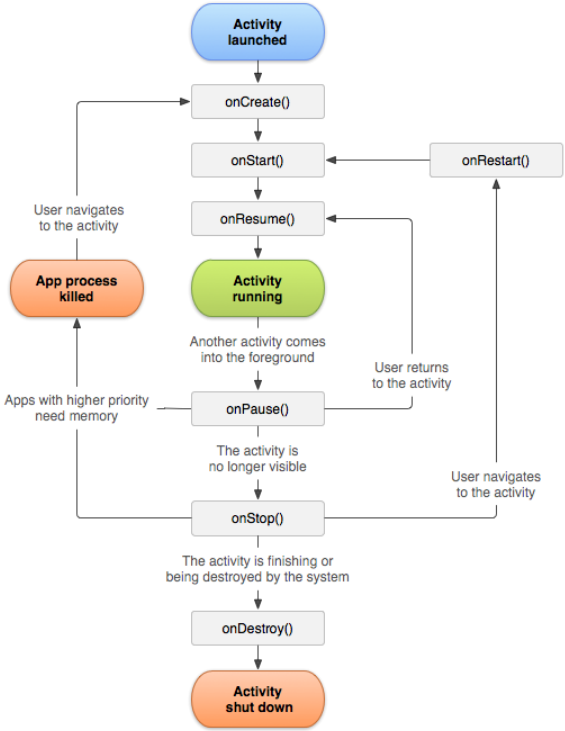
Estados

¿Qué pasa si giramos la pantalla?

3. Componentes
Estados

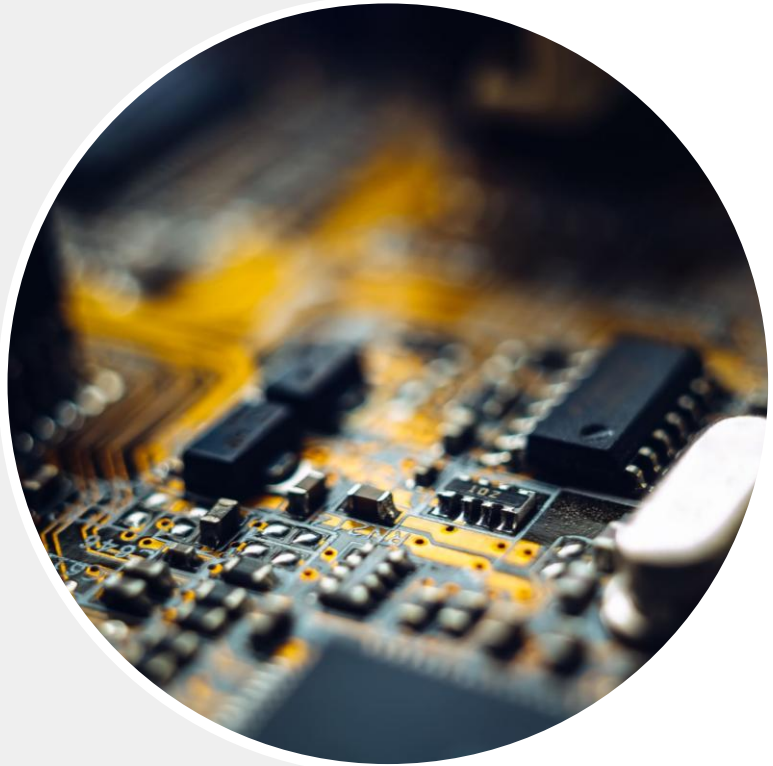
- Si giramos la pantalla se cambia la configuración
  - El ciclo de vida de la activity se destruye y se vuelve a dibujar
  - Todo vuelve al estado inicial
- Para evitarlo:

  rememberSaveable()



3.3. Componentes Material 3

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

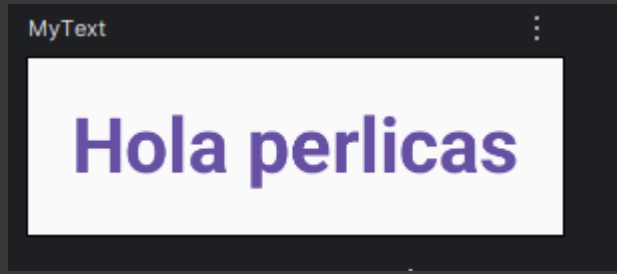
3. Componentes
Text

- Sirve para mostrar un texto por pantalla
- Propiedades más usadas:
  - text → el contenido del texto (string).
  - fontSize → tamaño de la letra (en sp).
  - color → color del texto.
  - fontWeight → grosor (ej. Bold, Normal).
  - textAlign → alineación (Center, Start, End).
  - maxLines → número máximo de líneas a mostrar.
  - overflow → qué hacer si el texto es muy largo (Ellipsis, cortar, etc.).
  - style → usar un estilo completo (TextStyle).

3. Componentes

Text

```
@Preview(showBackground = true)
@Composable
fun MyText() {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(
            text = "Hola perlicas",
            fontSize = 28.sp,
            fontWeight = FontWeight.Bold,
            color = MaterialTheme.colorScheme.primary
        )
    }
}
```



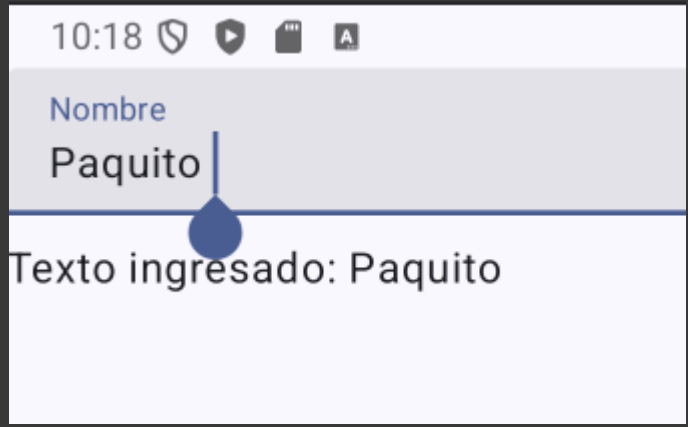
3. Componentes
TextField

- Un campo de texto editable que permite al usuario escribir información (ejemplo: nombre, correo, contraseña, búsqueda, etc.).
- Propiedades más usadas:
  - value → el texto actual del campo.
  - onValueChange → función que se ejecuta cada vez que el usuario escribe (para actualizar el estado).
  - label → texto guía que aparece encima (ej. “Correo”).
  - placeholder → texto dentro del campo mientras está vacío.
  - singleLine → restringe la entrada a una sola línea.
  - keyboardOptions → cambia el teclado (ej. tipo email, número, password).
  - leadingIcon / trailingIcon → iconos al inicio o al final del campo.

3. Componentes

TextField

```
@Composable
fun EjemploTextField(modifier: Modifier) {
    var texto by remember { mutableStateOf("") }
    Column(modifier = modifier) {
        TextField(
            value = texto,
            onValueChange = { texto = it },
            label = { Text("Nombre") },
            placeholder = { Text("Escribe tu nombre") },
            singleLine = true
        )
        Text(
            text = "Texto ingresado: $texto",
            modifier = Modifier.padding(top = 8.dp)
        )
    }
}
```



3. Componentes
TextField para contraseñas

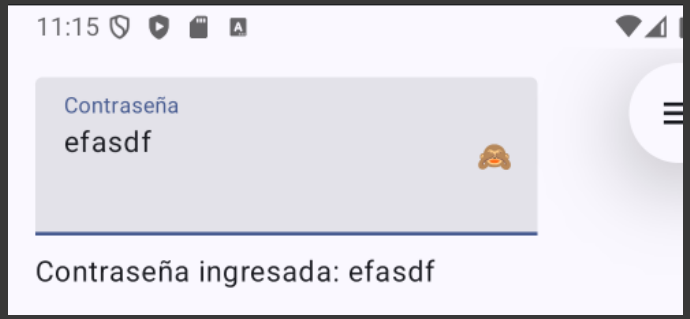
- Podemos aplicar un TextField para gestionar contraseñas
- Parámetros a considerar:
  - keyboardOptions-> muestra teclado adecuado (oculta predicciones/autocorrección).
  - visualTransformation-> define **cómo se ve el texto** mientras se escribe
  - trailingIcon-> añade un **icono al extremo derecho** del campo. Se usa para el **botón de "ver/ocultar"**.

3. Componentes

TextField para contraseñas

```
@Composable
fun MyPasswordTextField(modifier: Modifier) {
   var password by remember { mutableStateOf("") }
   var passwordVisible by remember { mutableStateOf(false) }
   Column(modifier = modifier.padding(16.dp)) {
      TextField(
         value = password,
         onValueChange = { password = it },
         label = { Text("Contraseña") },
         placeholder = { Text("Ingresa tu contraseña") },
         singleLine = true,
         visualTransformation = if (passwordVisible)
            VisualTransformation.None
         else
            PasswordVisualTransformation(),
         trailingIcon = {
            Text(text = if(passwordVisible) "🙈" else "👁️",
               modifier = modifier
                  .padding(8.dp)
                  .clickable { passwordVisible = !passwordVisible }
            )})}
```





3. Componentes
Button

- Es un componente interactivo para ejecutar una acción cuando el usuario lo toca
- **Parámetros**:
  - **onClick** → acción que ocurre cuando el botón es presionado.
  - **modifier** → permite aplicar padding, fillMaxWidth, height, etc.
  - **enabled** → si es false, el botón aparece deshabilitado y no responde.
  - **shape** → la forma del botón (por defecto redondeado).
  - **colors** → define los colores (fondo, texto, deshabilitado).
  - **elevation** → controla la sombra (cuando está elevado o presionado).
  - **contentPadding** → espacio entre el borde del botón y su contenido.
  - **interactionSource** → fuente de interacción para detectar estados como presionado, enfocado, etc.
  - **content** → lo que va dentro: Text, Icon, o ambos.

3. Componentes

Button

```
@Composable
fun MyButton(modifier: Modifier) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Button(
            onClick = {
                Log.e("MyButton", "Botón presionado" )
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Presionar")
        }
    }
}
```



3. Componentes
Imágenes

- Las imágenes se pueden mostrar de diferente manera:
  - Imágenes de recursos locales
    - Image()
  - Imágenes de URL:
    - Es necesario COIL (https://coil-kt.github.io/coil/)

3. Componentes

Ejemplo Image

```
@Composable
fun ImagenLocal(modifier: Modifier) {
    Image(
        painter = painterResource(id = R.drawable.images),
        contentDescription = "Mi imagen local",
        modifier = modifier.size(150.dp),
    )
}
```

Los recursos de imágenes se almacenan en res/drawables



3. Componentes
Imágenes librería COIL

- Instalar dependencia:
  - *implementation*("io.coil-kt:coil-compose:2.2.2")
- Dar permisos a la aplicación en el manifest:
  - <uses-permission android:name="android.permission.INTERNET"/>
- Parámetros importantes:
  - model → Lo más importante: puede ser una URL (String), un archivo (File), un recurso (R.drawable...).
  - contentDescription → Necesario para accesibilidad (si la imagen es decorativa, se puede poner null).

3. Componentes

Imágenes librería COIL

```
@Composable
fun ImagenURL(modifier: Modifier){
    AsyncImage(
        model = "https://picsum.photos/200/300",
        contentDescription = "Imagen de internet",
        modifier = modifier.size(200.dp),
    )
}
```



3. Componentes
Switch

- Es el componente que permite activar o desactivar una opción (encendido/apagado).
- Parámetros importantes:
  - checked → valor booleano (obligatorio).
  - onCheckedChange → callback cuando el valor cambia.
  - enabled → habilitar o deshabilitar el switch.
  - colors → personalizar colores con SwitchDefaults.colors().

3. Componentes

Switch

```
@Composable
fun MySwitch(modifier: Modifier) {
   var checked by remember { mutableStateOf(false) }

   Column(modifier = modifier.fillMaxSize(),
      verticalArrangement = Arrangement.Center,
      horizontalAlignment = Alignment.CenterHorizontally
   ){
      Switch(
         checked = checked,
         onCheckedChange = { checked = it }
      )
      Text(text = if (checked) "Activado" else "Desactivado")
   }
}
```



3. Componentes
Checkbox

- Es un componente de selección que permite al usuario **marcar o desmarcar** una opción
- Parámetros:
  - checked → valor booleano (obligatorio).
  - onCheckedChange → callback cuando cambia.
  - enabled → permite deshabilitar el checkbox.
  - colors → personalizar colores con CheckboxDefaults.colors().

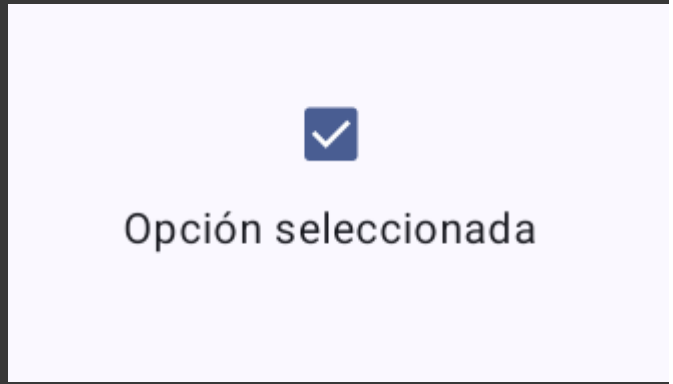
3. Componentes

Checkbox

```
@Composable
fun MyCheckbox(modifier: Modifier) {
    var checked by remember { mutableStateOf(false) }

    Column(modifier = modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ){
        Checkbox(
            checked = checked,
            onCheckedChange = { checked = it }
        )

        Text(text = if (checked) "Opción seleccionada" else "Opción no seleccionada")
    }
}
```



3. Componentes
RadioButton

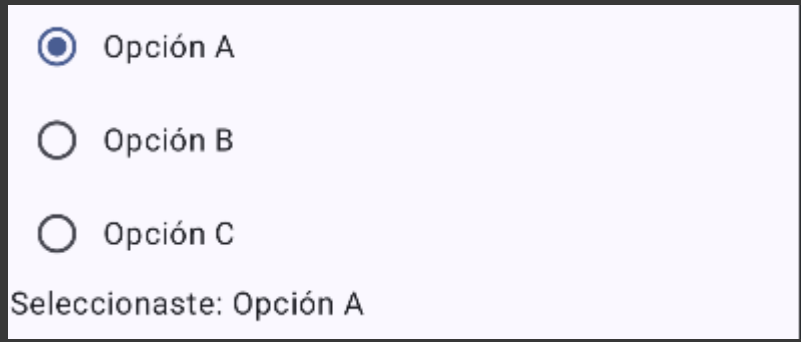
- Es un componente de selección que se usa cuando el usuario debe elegir **una sola opción entre varias** (selección exclusiva).
- Parámetros útiles de RadioButton
  - **selected** → booleano, indica si está marcado.
  - **onClick** → callback al seleccionarlo.
  - **enabled** → habilita o deshabilita.
  - **colors** → permite personalizar colores (RadioButtonDefaults.colors()).

3. Componentes

RadioButton

```
@Composable
fun MyRadioButton(modifier: Modifier) {
    val opciones = listOf("Opción A", "Opción B", "Opción C")
    var seleccion by remember { mutableStateOf(opciones[0]) }

    Column(modifier = modifier.padding(4.dp)) {
        opciones.forEach { opcion ->
            Row(
                verticalAlignment = Alignment.CenterVertically
            ) {
                RadioButton(
                    selected = (opcion == seleccion),
                    onClick = { seleccion = opcion }
                )
                Text(text = opcion)
            }
        }
        Text(text = "Seleccionaste: $seleccion")
    }
}
```



3. Componentes
Slider

- Es un componente interactivo que permite al usuario **seleccionar un valor dentro de un rango continuo**, moviendo un “control deslizante” de izquierda a derecha.
- Parámetros útiles de RadioButton
  - **value** → valor actual (Float).
  - **onValueChange** → acción al moverlo.
  - **valueRange** → rango de valores (0f..1f por defecto).
  - **steps** → define posiciones discretas.
  - **enabled** → habilitar o deshabilitar.
  - **colors** → personalizar colores con SliderDefaults.colors().

3. Componentes

Slider

```
@Composable
fun MySlider(modifier: Modifier) {
    var valor by remember { mutableStateOf(50f) }
    Column(modifier = modifier.padding(16.dp)) {
        Slider(
            value = valor,
            onValueChange = { valor = it },
            valueRange = 0f..100f,  // rango de 0 a 100
            steps = 9               // divide en 10 pasos (opcional)
        )
        Text(text = "Valor seleccionado: ${valor.toInt()}")
    }
}
```



3. Componentes
Dropdown

- Es un componente que muestra una **lista de opciones desplegable**, normalmente cuando el usuario toca un botón, un texto o un campo.
- Parámetros útiles de Dropdown y DropdownItem:
  - **DropdownMenu**

    expanded: Boolean → controla si está abierto.

    onDismissRequest: función para cerrarlo.

    modifier: tamaño y posición.

    offset: desplazar el menú respecto al elemento principal.
  - **DropdownMenuItem**

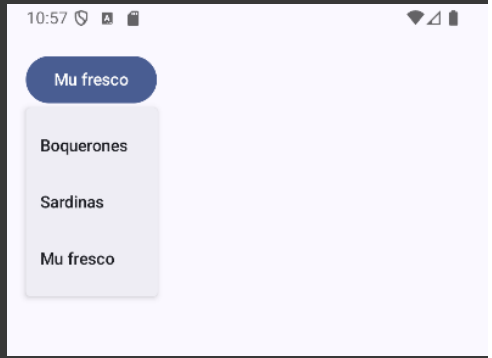
    text: contenido del ítem (normalmente un Text).

    onClick: acción al seleccionarlo.

    leadingIcon / trailingIcon: íconos opcionales.

3. Componentes
Dropdown

```
@Composable
fun MyDropdown(modifier: Modifier) {
    var expanded by remember { mutableStateOf(false) }    // controla si el menú está abierto
    val opciones = listOf("Boquerones", "Sardinas", "Mu fresco")
    var selectedOption by remember { mutableStateOf(opciones[0]) }
    Column(modifier = modifier.padding(16.dp)) {
        // Botón que abre o cierra el menú
        Button(onClick = { expanded = !expanded }) {
            Text(selectedOption)
        }
        // Menú desplegable
        DropdownMenu(
            expanded = expanded,
            onDismissRequest = { expanded = false } // se cierra al tocar fuera
        ) {
            opciones.forEach { opcion ->
                DropdownMenuItem(
                    text = { Text(opcion) },
                    onClick = {
                        selectedOption = opcion
                        expanded = false
                    })}}}}
```



3.3. Componentes del scaffold

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

3. Componentes

Componentes a situar en el scaffold

- El scaffold es un **contenedor base** que proporciona la **estructura visual principal** de una pantalla en una aplicación con **Material Design 3**.
- Componentes que se pueden situar dentro del Scaffold
  - **topBar**: contiene el título, íconos de navegación y acciones
  - **bottomBar**: sirve para moverse entre secciones principales de la app
  - floatingActionButton (FAB): se usa para la acción principal de la pantalla (por ejemplo, agregar, crear, enviar)
  - snackbarHost: se usa para confirmar acciones o mostrar errores menores.
  - **drawerContent**: se usa para mostrar opciones secundarias o de configuración.
  - content: aquí va el contenido dinámico: listas, formularios, texto, imágenes, etc.

3. Componentes
TopAppBar

- Es un componente de cabecera que se muestra en la parte superior de la pantalla y suele contener:
  - Un título
  - Un icono de navegación (por ejemplo, una flecha o un menú),
  - Y una o varias acciones (botones de búsqueda, ajustes, etc.).
- Parámetros importantes:
  - Title: composable con contenido principal
  - Navigation: icono a la izquierda
  - Actions: lista iconos o botones a la derecha
  - Modifier: modificador tamaño, color, etc.
  - colors:
  - scrollBehavior: animaciones desplazamiento

3. Componentes

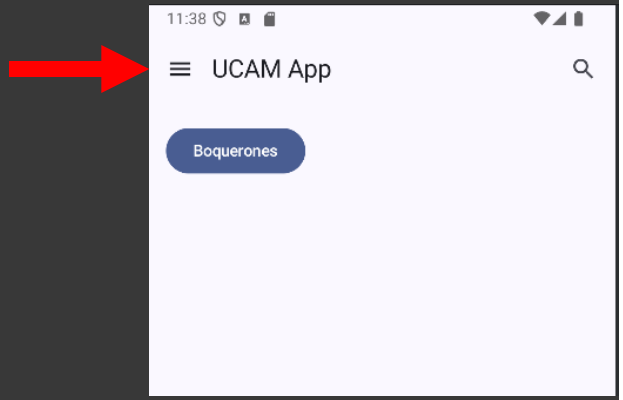
TopAppBar

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MyAppBar() {
   TopAppBar(
      title = { Text(“UCAM App") },
      navigationIcon = {
         IconButton(onClick = { /* abrir menú */ }) {
            Icon(Icons.Filled.Menu, contentDescription = "Menú")
         }
      },
      actions = {
         IconButton(onClick = { /* acción de búsqueda */ }) {
            Icon(Icons.Filled.Search, contentDescription = "Buscar")
         }
      }
   )
}
```

3. Componentes

TopAppBar en el Scaffold

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(
                    topBar = {MyAppBar()},
                    modifier = Modifier.fillMaxSize()) { innerPadding ->
                    MyDropdown(Modifier.padding(innerPadding))
                }
            }
        }
    }
}
```



3. Componentes
NavigationBar

- Es un **componente que permite al usuario moverse entre las principales secciones de una app**. Es la barra que normalmente ves al fondo de muchas aplicaciones (como YouTube, Instagram, Gmail, etc.), con **íconos y etiquetas** para cada sección.
- **Parámetros principales de NavigationBar**

  modifier → Permite ajustar tamaño, forma o aplicar padding.

  containerColor → Define el color de fondo de la barra.

  contentColor → Color del contenido (íconos y texto).

  tonalElevation → Controla la elevación o sombra de la barra.

  content → Contiene los elementos NavigationBarItem que conforman el menú.

3. Componentes
NavigationBar

- **Parámetros principales de NavigationBarItem**

  selected → Indica si el elemento está seleccionado (true o false).

  onClick → Acción que se ejecuta al presionar el ítem.

  icon → Ícono principal del elemento (obligatorio).

  label → Texto que acompaña al ícono (opcional).

  alwaysShowLabel → Define si el texto se muestra siempre o solo cuando el ítem está seleccionado.

  colors → Permite personalizar los colores del ítem (activo e inactivo).

3. Componentes

NavigationBar

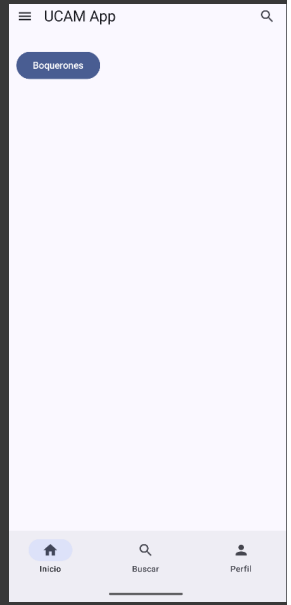
```
@Composable
fun MyNavBar() {
    var selectedItem by remember { mutableStateOf(0) }
    val items = listOf("Inicio", "Buscar", "Perfil")
    val icons = listOf(Icons.Filled.Home, Icons.Filled.Search, Icons.Filled.Person)

    NavigationBar {
        items.forEachIndexed { index, item ->
            NavigationBarItem(
                icon = { Icon(icons[index], contentDescription = item) },
                label = { Text(item) },
                selected = selectedItem == index,
                onClick = { selectedItem = index }
            )
        }
    }
}
```

3. Componentes

NavigationBar en el Scaffold

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(
                    topBar = {MyAppBar()},
                    bottomBar = { MyNavBar() },
                    modifier = Modifier.fillMaxSize()) { innerPadding ->
                    MyDropdown(Modifier.padding(innerPadding))
                }
            }
        }
    }
}
```



3. Componentes
NavigationDrawer

- Es uno de los componentes más útiles de Jetpack Compose (Material 3) cuando necesitas un menú lateral deslizante.
- Un ModalNavigationDrawer contiene:

  drawerContent → el contenido del panel lateral (menú).

  content → el contenido principal de la pantalla.

  drawerState → control de apertura con un **DrawerState** (abierto o cerrado).

3. Componentes

NavigationDrawer

```
@Composable
fun MyModalDrawer(content:@Composable () -> Unit) {
  val drawerState = rememberDrawerState(DrawerValue.Closed) //Necesario para controlar el estado del drawer
  val scope = rememberCoroutineScope() //Necesario para abrir y cerrar el drawer de forma asíncrona

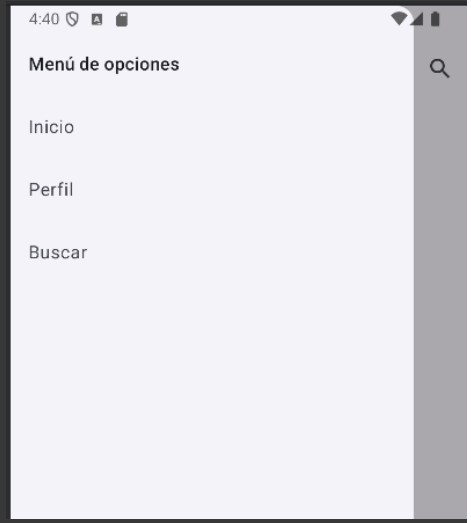
  ModalNavigationDrawer( //Contenedor del drawer
    drawerState = drawerState,
    drawerContent = {
      ModalDrawerSheet { //Contenido del drawer
        Text(
          text = "Menú de opciones",
          style = MaterialTheme.typography.titleMedium,
          modifier = Modifier.padding(16.dp)
        )
        NavigationDrawerItem( //Elemento del drawer
          label = { Text("Inicio") },
          selected = false,
          onClick = {
            scope.launch { drawerState.close() }
          }
        )
      }
    }
  ){
    content() //Contenido principal de la pantalla que se le pasa por parámetro
  }
}
```

3. Componentes

NavigationDrawer en el Scaffold

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                MyModalDrawer{ //se le pasa el content por parámetro
                    Scaffold(
                        topBar = {MyAppBar()},
                        bottomBar = { MyNavBar() },
                        modifier = Modifier.fillMaxSize()) { innerPadding ->
                        MyDropdown(Modifier.padding(innerPadding))
                    }
                }

            }
        }
    }
}
```



REFERENCIAS

BIBLIOGRÁFICAS

OpenCompose: https://www.jetpackcompose.pro/

Daniel Valero Carreras

dvalero@ucam.edu

UCAM Universidad Católica de Murcia



© UCAM